

VLSI Testing

Complete student lesson for course code **5493D**.

Revision 2026 Semester 5 Program Elective course 4 modules

Course snapshot

Scheme: 4 (L:4 T:0 P:0)

Credits: 4

Contact: 60 periods per semester

Module hours listed: 58

Official Source Declaration

This lesson uses the supplied official syllabus PDF as the authoritative source for course information, revision, modules, topics, objectives, Course Outcomes, assessment information, official activities, practical requirements, and references.

Source handling: Official wording and numerical values are retained wherever supplied. Educational explanations, Malayalam support, examples, diagrams, and revision questions are added as study support and are not presented as additional official requirements.

Transparency note: The supplied PDF does not specify a separate CIE/ESE distribution. Any limitation in the supplied PDF is not silently corrected.

How to Study This Lesson

Recommended sequence

1. Begin with the official source declaration and course dashboard.

2. Study each module in the official order and keep the coverage table as a checklist.
3. Open every topic, understand the example or procedure, and write your own short answer.
4. Use the revision section, glossary, questions, and practice paper after completing the modules.

Study principle: Keep English technical terminology, symbols, units, and official assessment wording unchanged in examination answers. Use the Malayalam support blocks only as a bridge to understanding.

Course Dashboard

Course code

5493D

Semester

5

Credits

4

Teaching scheme

4 (L:4 T:0 P:0)

Module-hour and outcome mapping

No.	Module	Official hours	Relevant CO / level
1	Introduction to VLSI Testing and Fault Models	11	CO1
2	Logic and Fault Simulation and Testability Measures	20	CO2
3	Test Generation for Combinational and Sequential Circuits	14	CO3
4	Design for Testability, BIST and Fault Diagnosis	13	CO4

Prerequisites

- Combinational Circuits — Digital Electronics — semester 3.
- Sequential Circuits and its types — Digital Electronics — semester 3.

How to study this lesson

1. Read the module introduction and learning goals.
2. Open every topic and reproduce its example, sequence, or procedure.
3. Use Malayalam support as a bridge; retain English technical terms for examinations.
4. Attempt the module questions without looking at the answers.

Objectives, Course Outcomes and CO-PO Information

Official course objectives

1. to introduce the VLSI testing
2. to introduce logic and fault simulation and testability measures
3. to study the test generation for combinational and sequential circuits
4. to study the design for testability.
5. to study the fault diagnosis

Course Outcomes

1. CO1: Understand VLSI Testing Process — 11 hours — Understanding.
2. CO2: Develop Logic Simulation and Fault Simulation — 20 hours — Understanding.
3. CO3: Develop Test for Combinational and Sequential Circuits — 14 hours — Understanding.
4. CO4: Understand the Design for Testability — 13 hours — Understanding.

CO-PO mapping as supplied

CO1: PO1 = 3 (Strongly mapped).

CO2: PO1 = 3 (Strongly mapped); PO2 = 2 (Moderately mapped).

CO3: PO1 = 3 (Strongly mapped); PO2 = 3 (Strongly mapped); PO3 = 3 (Strongly mapped).

CO4: PO1 = 3 (Strongly mapped); PO2 = 2 (Moderately mapped).

Legend supplied by the PDF: 3-Strongly mapped, 2-Moderately mapped, 1-Weakly mapped.

Complete Syllabus Coverage

Use this checklist to confirm that every official module topic is represented in the lesson.

Complete official topic coverage checklist

Module	Topic code	Topic	Hours
module-1	M1.01	Explain VLSI Testing Process	3
module-1	M1.02	Explain VLSI Testing Equipment	3
module-1	M1.03	Explain fault models	3
module-1	M1.04	Describe the relationship Among Fault Models.	2
module-2	M2.01	Explain Simulation for Design Verification and Test Evaluation	3
module-2	M2.02	Explain Modeling Circuits for Simulation	4
module-2	M2.03	Explain the algorithms for true value and fault simulation	6
module-2	M2.04	Find the faults in the given logic circuit	7
module-3	M3.01	Explain Combinational ATPG Algorithms	3
module-3	M3.02	Explain Sequential ATPG Algorithms	3
module-3	M3.03	Explain Simulation based ATPG	4
module-3	M3.04	Explain Genetic Algorithm Based ATPG	4
module-4	M4.01	Explain Testability basics and its analysis	4
module-4	M4.02	Explain BIST architecture	4
module-4	M4.03	Explain Fault Models for Diagnosis	5

Learning Roadmap

1. Introduction to VLSI Testing and Fault Models
CO1 · 11 hours

2. Logic and Fault Simulation and Testability Measures
CO2 · 20 hours

3. Test Generation for Combinational and Sequential Circuits
CO3 · 14 hours

4. Design for Testability, BIST and Fault Diagnosis
CO4 · 13 hours

The roadmap follows the order of the supplied syllabus.

OFFICIAL MODULE 1

Introduction to VLSI Testing and Fault Models

Testing is the evidence-based process of checking whether a manufactured VLSI circuit satisfies its intended behaviour and quality requirements. This module introduces the complete testing path, the equipment used to apply and observe tests, the economic consequences of defects, and the fault models that make complex physical defects manageable in digital analysis.

Hours: 11

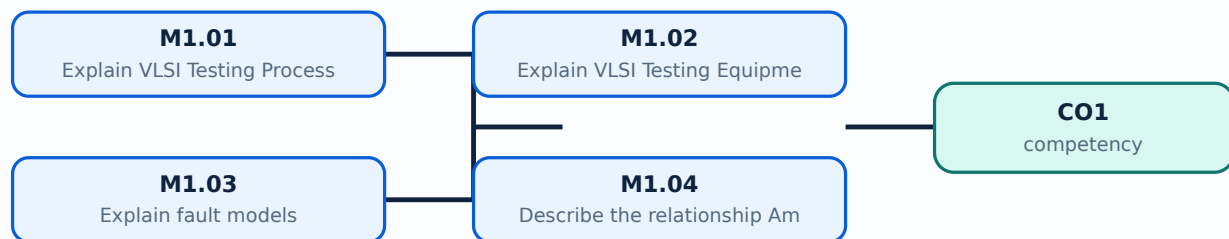
CO1 · Bloom level: Understanding

Learning goals

- Describe the VLSI testing process from design intent through production test and quality feedback.
- Identify the role of automatic test equipment and the main challenges in testing dense VLSI circuits.
- Explain common fault models and the relationship between physical defects and logical abstractions.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-1: the listed topics build the module competency.

Concept and explanation

The VLSI testing process converts a design specification into test requirements, derives test patterns, applies them to a device, captures the responses, compares the responses with expected values, and records pass/fail information. Testing may occur at wafer level, after packaging, during production screening, and during system-level diagnosis. A useful test flow separates defect prevention, design verification, manufacturing defect detection, quality grading, and failure analysis. Test coverage is the proportion of targeted faults detected by the applied test set; high coverage improves confidence but does not mean that every possible physical defect has been observed.

[illegible]

Study points

- Relate specification, test generation, pattern application, response capture, comparison, and diagnosis in the correct order.
- Distinguish design verification from manufacturing test.
- Define test coverage as a measure of detected targeted faults.

Engineering / workplace application: Semiconductor manufacturers use wafer probing, package testing, burn-in or screening, and failure analysis to maintain product quality across large production volumes.

Illustrative example: If a two-input gate is expected to implement AND, a test set must include input combinations that distinguish its correct truth table from likely stuck-at or bridging faults.

Common mistake: Treating functional simulation alone as a complete manufacturing test ignores defects introduced during fabrication and packaging.

– M1.02 — Explain VLSI Testing Equipment (3 hours)

Concept and explanation

Automatic Test Equipment (ATE) supplies controlled input patterns, timing, power, and measurement functions to a device under test. A typical setup includes a tester controller, pattern memory, timing and signal-generation resources, power supplies, pin electronics, a load board or probe card, and response-analysis hardware. Equipment selection is influenced by pin count, operating frequency, voltage levels, analogue or mixed-signal content, memory depth, measurement accuracy, and cost. The equipment must control signal integrity and protect the device while collecting repeatable results.

Malayalam support: ATE ഏതെങ്കിലും device under test-ന് patterns, timing, power നൽകി response അളക്കുന്ന test system ആണ്. Tester, pattern memory, timing unit, power supply, pin electronics, probe card എന്നിവയാണ് load board നൽകുന്ന ഏതെങ്കിലും ഏതെങ്കിലും.

Study points

- List the main functional blocks of an ATE setup.
- Explain why timing accuracy, pin count, voltage range, and measurement repeatability matter.
- Differentiate wafer probing from packaged-device testing at a high level.

Engineering / workplace application: ATE is used in wafer sort, final test, memory test, digital IC test, and production screening.

Illustrative example: A tester can apply a clocked pattern to a scan chain, capture the response, shift the captured bits out, and compare them with the expected signature.

Common mistake: Assuming that a low-cost logic analyser can replace production ATE overlooks timing, pin-parallelism, power, and automatic grading requirements.

– M1.03 — Explain fault models (3 hours)

Concept and explanation

A fault model is a simplified logical representation of a physical defect. Common digital models include stuck-at-0 and stuck-at-1 faults, open faults, short or bridging faults, transistor-level defects, and delay faults. The stuck-at model assumes that a signal line remains permanently at one logic value even when the fault-free circuit would produce the other value. A model is useful when it preserves the defect behaviours that matter for test generation without requiring every transistor-level detail in every calculation.

Malayalam support: Physical defect- $\square$ analysis $\square$ simplified logical representation $\square$ fault model. Stuck-at-0 signal $\square$ 0 $\square$; stuck-at-1 $\square$ 1 $\square$. Model $\square$ defect- $\square$ details $\square$.

Study points

- Define a fault model and explain why abstraction is needed.
- Compare stuck-at, bridging, open, and delay fault ideas.
- State that fault-free and faulty circuit responses must be distinguishable for detection.

Engineering / workplace application: Fault models guide ATPG tools, simulation experiments, coverage calculations, and diagnosis in digital VLSI production flows.

Illustrative example: To detect a line with a stuck-at-0 fault, a test must first justify the good circuit value of that line as 1 and then propagate the difference to an observable output.

Common mistake: A stuck-at fault is not the same as a logic value that happens to be 0 or 1 for one input pattern; it is a persistent abnormal constraint.

– M1.04 — Describe the relationship Among Fault Models. (2 hours)

Concept and explanation

Fault models are related by abstraction and by the defects they can represent. A physical defect may be represented by one or more logical faults, and a single logical model may cover several physical causes. For example, a short or transistor defect can sometimes produce a stuck-at behaviour under a particular circuit condition, while a delay defect may escape a purely static stuck-at test. The relationship is therefore one of useful coverage and equivalence under assumptions, not a claim that all models are identical.

Malayalam support: ഏകദേശം physical defect ന്റെ logical fault models ന്റെ represent ചെയ്യുന്നു. ഏകദേശം physical defect stuck-at behaviour ന്റെ; delay fault static test ന്റെ relationship ന്റെ. ഏകദേശം models ന്റെ complete equivalence ന്റെ, assumed behaviour ന്റെ relationship ന്റെ.

Study points

- Explain why one physical defect can map to multiple logical behaviours.
- Distinguish fault equivalence under a test model from physical identity.
- Relate static fault models to delay and defect-specific models.

Engineering / workplace application: Engineers select a model set according to product risk, test cost, timing requirements, and the defect mechanisms expected from a fabrication technology.

Illustrative example: A transition that reaches the correct final logic value but arrives too late may pass a static test and fail a timing-sensitive test.

Common mistake: Claiming that 100% stuck-at coverage guarantees 100% coverage of delay, bridging, or analogue defects is technically incorrect.

Module summary: Module 1 establishes the testing vocabulary: process, equipment, quality, abstraction, and the limits of each fault model.

Simulation predicts circuit behaviour before or during test evaluation. True-value simulation checks the fault-free response, while fault simulation injects selected faults and observes whether test patterns expose them at measurable outputs. The module also introduces Boolean Difference, path sensitization, and Kohavi-style reasoning for identifying faults in logic circuits.

Hours: 20

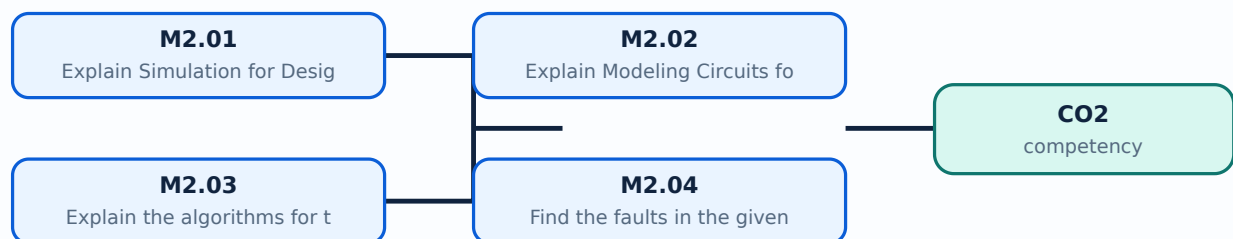
CO2 · Bloom level: Understanding

Learning goals

- Explain the purpose of simulation for design verification and test evaluation.
- Model logic circuits so that good and faulty responses can be compared.
- Apply the idea of fault activation and propagation to identify detected faults.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-2: the listed topics build the module competency.

– M2.01 — Explain Simulation for Design Verification and Test Evaluation (3 hours)

Concept and explanation

Simulation applies input vectors to a circuit model and computes the resulting internal and output values. In design verification, the objective is to check whether the implementation follows the specification. In test evaluation, the objective is to determine whether a test pattern detects a targeted fault. A good simulation flow defines inputs and clocking, establishes expected responses, runs the fault-free model, introduces faults when required, and records mismatches, coverage, and unresolved cases.

Malayalam support: Simulation input vectors circuit-ഉത്തരവ് output response ഉത്തരവ്. Design verification specification ഉത്തരവ് ഉത്തരവ്; test evaluation test pattern fault ഉത്തരവ് ഉത്തരവ്. Good model-ഉത്തരവ് faulty model-ഉത്തരവ് compare ഉത്തരവ്.

Study points

- Separate fault-free simulation from fault simulation.
- Describe stimulus, response, comparison, and coverage as parts of a simulation flow.
- Explain why a mismatch must be traced to an input, model, timing assumption, or fault.

Engineering / workplace application: HDL simulation, regression testing, and production-test pattern validation reduce the chance that an incorrect test sequence reaches silicon.

Illustrative example: For input vector 10, simulate the good circuit and a line stuck-at-1 circuit; if the output differs, the vector detects that fault.

Common mistake: A waveform that looks different is not automatically a detected fault unless the fault-free expected response and observation point are defined.

– M2.02 — Explain Modeling Circuits for Simulation (4 hours)

Concept and explanation

A simulation model represents gates, interconnections, storage elements, delays, and external signals at a selected abstraction level. A combinational model maps current inputs to outputs; a sequential model also represents state, clock events, reset, and feedback. A useful test model has clear signal names, deterministic initial conditions, correct logic polarity, and a testbench that applies meaningful patterns. Modelling resolution matters because a gate-level model, switch-level model, and behavioural model expose different details and computational costs.

Malayalam support: Simulation model circuit-കാലക്രമം gates, wires, storage, delays, inputs ക്രമത്തിൽ represent ക്രമത്തിൽ. Combinational model current inputs ക്രമത്തിൽ ക്രമത്തിൽ; sequential model state, clock, reset ക്രമത്തിൽ ക്രമത്തിൽ. Abstraction level ക്രമത്തിൽ detail ക്രമത്തിൽ.

Study points

- Choose a model level appropriate to the verification or testing question.
- Include state initialisation, clocking, reset, and signal polarity in a sequential model.
- Explain why model assumptions must be documented.

Engineering / workplace application: Reusable gate models and testbenches support regression testing, fault injection, and comparison of alternative implementations.

Illustrative example: A simple combinational testbench may cycle through all four inputs of a two-input gate, whereas a sequential testbench must also define reset and clock transitions.

Common mistake: Leaving an uninitialised sequential state undocumented can produce X values or misleading fault conclusions.

– M2.03 — Explain the algorithms for true value and fault simulation (6 hours)

Concept and explanation

True-value simulation evaluates the fault-free circuit for each input vector. Fault simulation repeats or augments that computation for circuits containing targeted faults. A fault is activated when the good value at its site differs from the faulty value; it is propagated when a sensitised path carries that difference to a primary output or scan observation point. Fault lists can be reduced using equivalence or dominance assumptions, and detected faults can be dropped from later simulation. The Boolean Difference Method relates output sensitivity to an input or internal variable; path sensitization selects non-controlling values on side inputs so the fault effect can pass through a gate; the Kohavi Algorithm is an algorithmic procedure for logic-fault analysis and test generation.

Malayalam support: Good circuit value V_g faulty circuit value V_f compare $V_g \neq V_f$ fault simulation. Fault site- s values $V_g(s) \neq V_f(s)$ activation; output V_o difference $V_o \neq V_o$ propagation. Boolean Difference sensitivity $\frac{\partial V_o}{\partial V_i}$; path sensitization side inputs non-controlling values $V_i = 0$.

Study points

- Define activation and propagation.
- Explain fault dropping, equivalence, and dominance as ways to reduce computation.
- State the roles of Boolean Difference Method, path sensitization method, and Kohavi Algorithm.

Engineering / workplace application: ATPG and fault-simulation tools use these ideas to evaluate large fault lists without manually analysing every transistor.

Illustrative example: For an AND gate, a fault effect on one input can propagate only when the other input is at the non-controlling value 1; a 0 on the other input masks the difference.

Common mistake: Activating a fault is not sufficient; the effect must also propagate to an observable point.

– M2.04 — Find the faults in the given logic circuit (7 hours)

Concept and explanation

Finding faults in a logic circuit follows a repeatable sequence. Identify the fault site and fault type, determine the good value required to activate it, choose values on side inputs that avoid masking, propagate the difference through a path, and observe the output. If no path can be sensitised, the fault may be redundant for the chosen observation or equivalent to another fault. For sequential logic, the procedure also includes state initialisation, clock sequencing, and observation over time. A fault table records the vector, expected good output, faulty output, and detection status.

Malayalam support: Circuit- $\square$ fault $\square\square\square\square\square\square\square\square$ $\square\square\square\square$ fault activate $\square\square\square\square\square\square$, $\square\square\square\square\square\square$ side inputs $\square\square\square\square\square\square\square\square$ effect output $\square\square$ propagate $\square\square\square\square\square\square$, $\square\square\square\square\square$ output observe $\square\square\square\square\square\square$. Sequential circuit- $\square$ state, clock sequence $\square\square\square\square\square\square\square\square$ $\square\square\square\square\square\square\square\square\square\square$.

Study points

- Construct a good/faulty response table for a small combinational circuit.
- Use controlling and non-controlling values to reason about propagation.
- Record detected, undetected, redundant, and possibly equivalent faults separately.

Engineering / workplace application: Fault tables are useful in laboratory exercises, ATPG debugging, design-for-test reviews, and failure-analysis reports.

Illustrative example: For a two-level AND-OR network, select a vector that makes the target branch active and every side input on the propagation path non-controlling, then compare the final output.

Common mistake: Skipping the good-circuit response makes it impossible to prove that the observed output difference is caused by the injected fault.

Module summary: Module 2 turns fault models into observable evidence through good-value simulation, faulty-value simulation, activation, propagation, and testability reasoning.

Test Generation for Combinational and Sequential Circuits

Automatic Test Pattern Generation (ATPG) searches for input or input-sequence patterns that detect specified faults. The search must represent circuit values and fault conditions, identify redundant logic, and handle the difference between memoryless combinational logic and state-dependent sequential logic.

Hours: 14

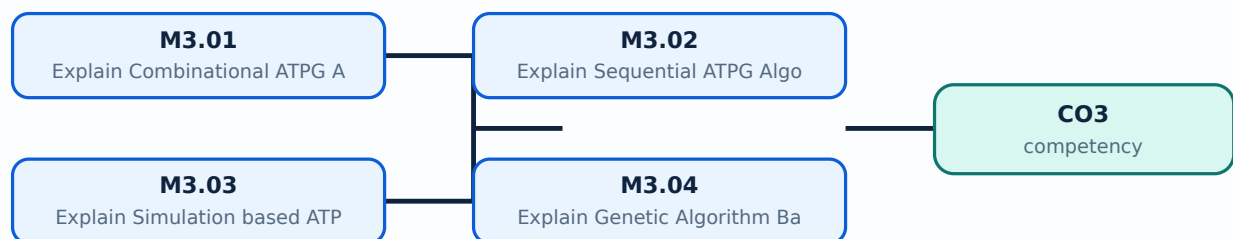
CO3 · Bloom level: Understanding

Learning goals

- Describe the representations and objectives used by ATPG algorithms.
- Differentiate combinational and sequential test-generation problems.
- Explain simulation-based and genetic-algorithm-based approaches at a conceptual level.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-3: the listed topics build the module competency.

– M3.01 — Explain Combinational ATPG Algorithms (3 hours)

Concept and explanation

Combinational ATPG finds an input vector that activates a fault and propagates its effect to a primary output. Common reasoning steps are fault activation, forward implication, backward implication, path sensitization, and conflict resolution. Algorithms may use circuit topology, logic values, D-algebra or related symbolic representations, and systematic backtracking. A valid result must satisfy both the activation condition and the observation condition; if every possible path is blocked, the fault is redundant or untestable under the model.

Malayalam support: Combinational ATPG- $\square$ input vector $\square\square\square\square\square\square\square\square$ fault activate $\square\square\square\square\square\square\square\square$ output $\square\square$ propagate $\square\square\square\square\square\square\square\square$ $\square\square\square$. Forward implication, backward implication, path sensitization, conflict resolution $\square\square\square\square\square$ $\square\square\square\square\square$ steps $\square\square$.

Study points

- State the two essential conditions for detecting a combinational fault.
- Explain the purpose of implication and backtracking.
- Identify redundant or untestable faults conceptually.

Engineering / workplace application: Combinational ATPG creates production test patterns for logic blocks, arithmetic units, control logic, and scan-test combinational portions.

Illustrative example: If a target line must be 1 for activation and a downstream OR gate needs its other input 0 for propagation, ATPG assigns both conditions before checking upstream consistency.

Common mistake: A partial assignment that satisfies activation but causes a propagation conflict is not a completed ATPG solution.

– M3.02 — Explain Sequential ATPG Algorithms (3 hours)

Concept and explanation

Sequential ATPG must control and observe circuit state as well as primary inputs. A test sequence may require an initialisation or reset operation, several capture cycles, and a final observation cycle. The algorithm searches through state transitions, uses time-frame expansion or state justification, and accounts for feedback and storage elements. Scan design simplifies the problem by making internal state directly loadable and observable; without scan, the required sequence may be longer and the search space much larger.

Malayalam support: Sequential ATPG-ൽ input ഫീഡ്ബാക്ക്; internal state initialize ചെയ്ത ശേഷം clock cycles ൽ fault effect propagate ചെയ്ത ശേഷം. Scan design state load/observe ചെയ്ത ശേഷം ഫോൾട്ട് ഡെറ്റക്ട് ചെയ്യാം.

Study points

- Explain why a sequential test is a sequence rather than one vector.
- Include reset, state justification, capture, and observation in the reasoning.
- Compare the difficulty of scan and non-scan sequential test generation.

Engineering / workplace application: Microcontrollers, controllers, pipelines, and finite-state machines use sequential ATPG or scan-assisted ATPG to detect storage and transition-related defects.

Illustrative example: A sequence may reset a state machine, drive it to a target state, apply a sensitising input, and capture the faulty transition at the next clock edge.

Common mistake: Applying a combinational ATPG vector at one instant does not prove that a sequential fault is detected across clocked state transitions.

— M3.03 — Explain Simulation based ATPG (4 hours)

Concept and explanation

Simulation-based ATPG uses circuit simulation to evaluate candidate test patterns and guide the search toward patterns that detect remaining faults. It may begin with random or deterministic patterns, simulate the fault list, drop detected faults, and generate additional patterns for faults that remain undetected. The method is simple to integrate with existing simulators and can quickly detect easy faults, but it may require many patterns for hard-to-detect faults and does not guarantee a minimum test set.

Malayalam support: Simulation-based ATPG candidate patterns simulate $\square\square\square\square\square$ detected faults remove $\square\square\square\square\square\square\square\square$. Remaining hard faults- $\square\square\square\square$ $\square\square\square\square$ patterns $\square\square\square\square\square\square\square\square\square\square$. Easy faults $\square\square\square\square\square\square$ $\square\square\square\square\square\square\square\square$; hard faults- $\square\square$ $\square\square\square\square\square$ patterns $\square\square\square\square\square\square\square$.

Study points

- Describe fault dropping in a simulation-based ATPG loop.
- Compare random stimulation with deterministic pattern generation.
- State the trade-off between simplicity, pattern count, and guaranteed coverage.

Engineering / workplace application: Simulation-based pattern generation is useful for early verification, fault-list screening, and supplementing deterministic ATPG patterns.

Illustrative example: After simulating a batch of random vectors, the tool removes detected stuck-at faults and focuses deterministic search on the remaining list.

Common mistake: A large number of random patterns does not automatically mean high coverage of rare or sequentially difficult faults.

– M3.04 — Explain Genetic Algorithm Based ATPG (4 hours)

Concept and explanation

A Genetic Algorithm (GA) represents a candidate test pattern or sequence as a chromosome. A population of candidates is evaluated by a fitness function such as the number of faults detected, fault-effect propagation, or coverage gain. Selection retains stronger candidates; crossover combines portions of parent patterns; mutation introduces variation. The process repeats until a coverage target, iteration limit, or acceptable fitness is reached. GA-based ATPG can explore a large search space, but it is heuristic and needs careful encoding, fitness design, and reproducible stopping criteria.

Malayalam support: GA-based ATPG-ൽ test pattern chromosome ന്നത് represent ന്നതാണ്. Fitness function detected faults/coverage ന്നതാണ്. Selection, crossover, mutation ന്നത് ന്നതാണ് patterns ന്നതാണ് search ന്നതാണ്. ന്നത് heuristic method ന്നത്.

Study points

- Identify chromosome, population, fitness, selection, crossover, and mutation.
- Explain why the fitness function must reward both activation and propagation.
- State that a heuristic result is not automatically optimal or complete.

Engineering / workplace application: Heuristic ATPG is relevant when exact search is expensive, especially for complex logic networks or long sequential sequences.

Illustrative example: If a chromosome encodes eight input bits, crossover can exchange a high-order segment between two parent patterns and mutation can flip one selected bit before resimulation.

Common mistake: Using only output switching as the fitness function may produce active patterns that never expose the target fault at an observation point.

Module summary: Module 3 compares systematic and heuristic test-generation approaches for memoryless and stateful digital circuits.

Design for Testability (DFT) adds structures and design practices that make internal nodes easier to control and observe. Scan chains and Built-In Self-Test (BIST) reduce the difficulty of applying external patterns, while diagnosis uses response information to locate likely fault sites rather than merely declaring pass or fail.

Hours: 13

CO4 · Bloom level: Understanding

Learning goals

- Explain controllability, observability, testability analysis, and scan structures.
- Describe BIST architecture and Random Logic BIST at a block level.
- Differentiate fault detection from fault diagnosis and identify diagnostic vector roles.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-4: the listed topics build the module competency.

– M4.01 — Explain Testability basics and its analysis (4 hours)

Concept and explanation

Testability describes how easily a circuit can be controlled from available inputs and how easily internal effects can be observed at outputs. Controllability measures the effort needed to set a node to a required logic value; observability measures the effort needed to propagate a node effect to an observation point. Testability analysis identifies hard-to-control, hard-to-observe, reconvergent, redundant, or highly sequential regions. Scan cell designs replace or augment storage elements so internal state can be shifted in and out, and scan architecture connects these cells into one or more scan chains.

Malayalam support: Controllability node-ന് 0/1 ന്റെ ആവശ്യമായ നിലയിലാക്കാനുള്ള ശ്രമം അളക്കുന്നു. Observability internal fault effect output-ന് എത്തിച്ചേരാനുള്ള ശ്രമം അളക്കുന്നു. Scan chain internal flip-flops serial ന്റെ load/shift ന്റെ ആവശ്യമായ ശ്രമം അളക്കുന്നു.

Study points

- Define controllability and observability.
- Explain why reconvergent paths and storage elements create testability difficulty.
- Describe scan cell and scan-chain operation at a block level.

Engineering / workplace application: DFT analysis guides scan insertion, test-point selection, compression planning, and ATPG feasibility in complex SoCs.

Illustrative example: A scan chain can shift a known state into internal flip-flops, apply a capture cycle, and shift the captured response out for comparison.

Common mistake: High controllability alone does not guarantee high testability; the resulting fault effect must also be observable.

– M4.02 — Explain BIST architecture (4 hours)

Concept and explanation

Built-In Self-Test (BIST) places test-pattern generation, response compaction or analysis, control, and pass/fail decision logic on or near the chip. In Random Logic BIST, a pseudo-random pattern generator stimulates logic and a signature register compacts responses; an on-chip controller sequences reset, pattern application, capture, and completion. BIST reduces dependence on external tester pins and can support field or power-on test, but it consumes area, power, design effort, and may suffer from aliasing or random-pattern-resistant faults.

Malayalam support: BIST chip-ഉള്ള test pattern generator, response analyzer, controller, pass/fail logic ഉൾപ്പെടെയുള്ളവ. Random Logic BIST pseudo-random patterns generate ഉള്ള signature register response compact ഉള്ള ഉള്ള.

Study points

- Identify pattern generation, response compaction, control, and decision blocks.
- Explain the use of pseudo-random patterns and a signature register.
- State BIST benefits and limitations, including area, power, and aliasing.

Engineering / workplace application: Memory BIST, processor self-test, automotive electronics, and safety-critical controllers use on-chip test structures for startup or maintenance checks.

Illustrative example: A BIST controller can apply a fixed number of pseudo-random patterns, compact each response into a Multiple-Input Signature Register, and compare the final signature.

Common mistake: A compacted signature matching the expected value is strong evidence, but signature aliasing means different responses can occasionally produce the same signature.

– M4.03 — Explain Fault Models for Diagnosis (5 hours)

Concept and explanation

Fault diagnosis goes beyond detection: it uses failing responses to identify or rank possible fault locations and types. Diagnostic fault models may include stuck-at, bridging, transition, open, or scan-chain faults according to the observed failure mechanism. Diagnosis begins with basic definitions and a candidate fault list, generates distinguishing vectors, applies or simulates those vectors, and compares response signatures. Combinational logic diagnosis uses output differences and structural reasoning; Scan Chain Diagnosis examines shift and capture behaviour; Logic BIST diagnosis uses signatures, checkpoints, or additional diagnostic patterns to narrow the suspect region.

Malayalam support: Fault detection chip faulty ഫോൾട്ട് ഫോൾട്ട് ഫോൾട്ട്; diagnosis fault ഡയഗ്നോസിക് ഫോൾട്ട് ഡയഗ്നോസിക് ഫോൾട്ട് ഡയഗ്നോസിക് ഫോൾട്ട്. Diagnostic vectors candidate faults ഡയഗ്നോസിക് ഫോൾട്ട് ഡയഗ്നോസിക് ഫോൾട്ട്. Scan chain- shift/capture response ഡയഗ്നോസിക് ഫോൾട്ട് ഡയഗ്നോസിക് ഫോൾട്ട്.

Study points

- Define diagnosis and distinguish it from detection.
- Explain diagnostic vector generation and candidate-fault reduction.
- Compare combinational logic, scan-chain, and Logic BIST diagnosis.

Engineering / workplace application: Failure analysis, yield improvement, service troubleshooting, and safety monitoring use diagnosis to reduce repair time and identify recurring process defects.

Illustrative example: If two candidate faults produce the same response for one vector, apply a second vector selected so that their predicted responses differ; the result eliminates one candidate.

Common mistake: A single failing output usually identifies a symptom, not a unique physical fault; multiple distinguishing patterns are often needed.

Module summary: Module 4 makes internal circuitry testable and uses structured response information to detect, localise, and diagnose faults.

Formula and Notation Bank

Formula note: The supplied syllabus is primarily procedural or conceptual and does not prescribe a compulsory numerical formula bank. Focus on the official terminology, sequences, records, and practical demonstrations listed in the modules.

Diagram Library for Rapid Revision

[Click here to view the labelled learning-map diagram.](#)

[Click here to view the labelled learning-map diagram.](#)

[Click here to view the labelled learning-map diagram.](#)

[Click here to view the labelled learning-map diagram.](#)

[Click here to view the labelled learning-map diagram.](#)

[Click here to view the labelled learning-map diagram.](#)

[Click here to view the labelled learning-map diagram.](#)

[Open the module to view its labelled learning-map diagram.](#)

[Module 2: Logic and Fault](#)

[Module 3: Test Generation](#)

[Module 4: Design for](#)

Official Activities and Practical Requirements

Official activities / self-learning

- The supplied syllabus does not provide a separate official self-learning activity list for this theory elective.
- For study support, draw one fault model or scan/BIST block diagram after each relevant topic and explain the signal path in your own words.
- Use the worked examples and module questions as revision activities; these are educational additions, not official assessment requirements.

Practical requirements / equipment

- The supplied syllabus does not prescribe separate practical equipment for this theory elective.
- For optional practice, a digital-logic simulator or HDL testbench may be used to observe fault effects; the supplied PDF does not mandate a particular software package.

Assessment Methodology

The supplied PDF does not specify a separate CIE/ESE distribution.

- The supplied PDF states “Series Tests 2” in the course-outcome block and separately lists Series Test 1 — 1 hour after Module 2 and Series Test 2 — 1 hour after Module 4.
- A separate CIE/ESE mark distribution, attendance component, and official question-paper pattern are not specified in the supplied PDF; no marks or attendance values have been invented here.
- The official module durations total 58 hours; the two separately listed one-hour series tests account for the 60 periods per semester.

Module-wise Questions and Answers

module-1 — Introduction to VLSI Testing and Fault Models

1. Explain Explain VLSI Testing Process. [3 marks]

The VLSI testing process converts a design specification into test requirements, derives test patterns, applies them to a device, captures the responses, compares the responses with expected values, and records pass/fail information. Testing may occur at wafer level, after packaging, during production screening, and during system-level diagnosis. A useful test flow separates defect prevention, design verification, manufacturing defect detection, quality grading, and failure analysis. Test coverage is the proportion of targeted faults detected by the applied test set; high coverage improves confidence but does not mean that every possible physical defect has been observed.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

2. Explain Explain VLSI Testing Equipment. [3 marks]

Automatic Test Equipment (ATE) supplies controlled input patterns, timing, power, and measurement functions to a device under test. A typical setup includes a tester controller, pattern memory, timing and signal-generation resources, power supplies, pin electronics, a load board or probe card, and response-analysis hardware. Equipment selection is influenced by pin count, operating frequency, voltage levels, analogue or mixed-signal content, memory

depth, measurement accuracy, and cost. The equipment must control signal integrity and protect the device while collecting repeatable results.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

3. Explain Explain fault models. [3 marks]

A fault model is a simplified logical representation of a physical defect. Common digital models include stuck-at-0 and stuck-at-1 faults, open faults, short or bridging faults, transistor-level defects, and delay faults. The stuck-at model assumes that a signal line remains permanently at one logic value even when the fault-free circuit would produce the other value. A model is useful when it preserves the defect behaviours that matter for test generation without requiring every transistor-level detail in every calculation.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

4. Explain Describe the relationship Among Fault Models.. [3 marks]

Fault models are related by abstraction and by the defects they can represent. A physical defect may be represented by one or more logical faults, and a single logical model may cover several physical causes. For example, a short or transistor defect can sometimes produce a stuck-at behaviour under a particular circuit condition, while a delay defect may escape a purely static stuck-at test. The relationship is therefore one of useful coverage and equivalence under assumptions, not a claim that all models are identical.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-2 — Logic and Fault Simulation and Testability Measures

5. Explain Explain Simulation for Design Verification and Test Evaluation. [3 marks]

Simulation applies input vectors to a circuit model and computes the resulting internal and output values. In design verification, the objective is to check whether the implementation follows the specification. In test evaluation, the objective is to determine whether a test pattern detects a targeted fault. A good simulation flow defines inputs and clocking, establishes expected responses, runs the fault-free model, introduces faults when required, and records mismatches, coverage, and unresolved cases.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

6. Explain Explain Modeling Circuits for Simulation. [3 marks]

A simulation model represents gates, interconnections, storage elements, delays, and external signals at a selected abstraction level. A combinational model maps current inputs to outputs; a sequential model also represents state, clock events, reset, and feedback. A useful test model has clear signal names, deterministic initial conditions, correct logic polarity, and a testbench that applies meaningful patterns. Modelling resolution matters because a gate-level model, switch-level model, and behavioural model expose different details and computational costs.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

7. Explain Explain the algorithms for true value and fault simulation. [3 marks]

True-value simulation evaluates the fault-free circuit for each input vector. Fault simulation repeats or augments that computation for circuits containing targeted faults. A fault is activated when the good value at its site differs from the faulty value; it is propagated when a sensitised path carries that difference to a primary output or scan observation point. Fault lists can be reduced using equivalence or dominance assumptions, and detected faults can be dropped from later simulation. The Boolean Difference Method relates output sensitivity to an input or internal variable; path sensitization selects non-controlling values on side inputs so the fault effect can pass through a gate; the Kohavi Algorithm is an algorithmic procedure for logic-fault analysis and test generation.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

8. Explain Find the faults in the given logic circuit. [3 marks]

Finding faults in a logic circuit follows a repeatable sequence. Identify the fault site and fault type, determine the good value required to activate it, choose values on side inputs that avoid masking, propagate the difference through a path, and observe the output. If no path can be sensitised, the fault may be redundant for the chosen observation or equivalent to another fault. For sequential logic, the procedure also includes state initialisation, clock sequencing, and observation over time. A fault table records the vector, expected good output, faulty output, and detection status.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-3 — Test Generation for Combinational and Sequential Circuits

9. Explain Explain Combinational ATPG Algorithms. [3 marks]

Combinational ATPG finds an input vector that activates a fault and propagates its effect to a primary output. Common reasoning steps are fault activation, forward implication, backward implication, path sensitization, and conflict resolution. Algorithms may use circuit topology, logic values, D-algebra or related symbolic representations, and systematic backtracking. A valid result must satisfy both the activation condition and the observation condition; if every possible path is blocked, the fault is redundant or untestable under the model.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

10. Explain Explain Sequential ATPG Algorithms. [3 marks]

Sequential ATPG must control and observe circuit state as well as primary inputs. A test sequence may require an initialisation or reset operation, several capture cycles, and a final observation cycle. The algorithm searches through state transitions, uses time-frame expansion or state justification, and accounts for feedback and storage elements. Scan design simplifies the problem by making internal state directly loadable and observable; without scan, the required sequence may be longer and the search space much larger.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

11. Explain Explain Simulation based ATPG. [3 marks]

Simulation-based ATPG uses circuit simulation to evaluate candidate test patterns and guide the search toward patterns that detect remaining faults. It may begin with random or deterministic patterns, simulate the fault list, drop detected faults, and generate additional patterns for faults that remain undetected. The method is simple to integrate with existing simulators and can quickly detect easy faults, but it may require many patterns for hard-to-detect faults and does not guarantee a minimum test set.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

12. Explain Explain Genetic Algorithm Based ATPG. [3 marks]

A Genetic Algorithm (GA) represents a candidate test pattern or sequence as a chromosome. A population of candidates is evaluated by a fitness function such as the number of faults detected, fault-effect propagation, or coverage gain. Selection retains stronger candidates; crossover combines portions of parent patterns; mutation introduces variation. The process repeats until a coverage target, iteration limit, or acceptable fitness is reached. GA-based ATPG can explore a large search space, but it is heuristic and needs careful encoding, fitness design, and reproducible stopping criteria.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-4 — Design for Testability, BIST and Fault Diagnosis

13. Explain Explain Testability basics and its analysis. [3 marks]

Testability describes how easily a circuit can be controlled from available inputs and how easily internal effects can be observed at outputs. Controllability measures the effort needed to set a node to a required logic value; observability measures the effort needed to propagate a node effect to an observation point. Testability analysis identifies hard-to-control, hard-to-observe, reconvergent, redundant, or highly sequential regions. Scan cell designs replace or augment storage elements so internal state can be shifted in and out, and scan architecture connects these cells into one or more scan chains.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

14. Explain Explain BIST architecture. [3 marks]

Built-In Self-Test (BIST) places test-pattern generation, response compaction or analysis, control, and pass/fail decision logic on or near the chip. In Random Logic BIST, a pseudo-random pattern generator stimulates logic and a signature register compacts responses; an on-chip controller sequences reset, pattern application, capture, and completion. BIST reduces dependence on external tester pins and can support field or power-on test, but it consumes area, power, design effort, and may suffer from aliasing or random-pattern-resistant faults.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

15. Explain Explain Fault Models for Diagnosis. [3 marks]

Fault diagnosis goes beyond detection: it uses failing responses to identify or rank possible fault locations and types. Diagnostic fault models may include stuck-at, bridging, transition, open, or scan-chain faults according to the observed failure mechanism. Diagnosis begins with basic definitions and a candidate fault list, generates distinguishing vectors, applies or simulates those vectors, and compares response signatures. Combinational logic diagnosis uses output differences and structural reasoning; Scan Chain Diagnosis examines shift and capture behaviour; Logic BIST diagnosis uses signatures, checkpoints, or additional diagnostic patterns to narrow the suspect region.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

Practice Model Question Paper

Practice Model Paper — Not an Official Examination Paper

This paper is generated from the supplied module coverage for revision and does not claim to reproduce the official examination pattern.

1. **1.** Define or explain *Explain VLSI Testing Process* with one relevant application. (5 marks)
2. **2.** Define or explain *Explain VLSI Testing Equipment* with one relevant application. (5 marks)
3. **3.** Define or explain *Explain Simulation for Design Verification and Test Evaluation* with one relevant application. (5 marks)
4. **4.** Define or explain *Explain Modeling Circuits for Simulation* with one relevant application. (5 marks)
5. **5.** Define or explain *Explain Combinational ATPG Algorithms* with one relevant application. (5 marks)
6. **6.** Define or explain *Explain Sequential ATPG Algorithms* with one relevant application. (5 marks)
7. **7.** Define or explain *Explain Testability basics and its analysis* with one relevant application. (5 marks)
8. **8.** Define or explain *Explain BIST architecture* with one relevant application. (5 marks)

Writing advice: Use headings, standard terminology, and labelled diagrams or procedures where relevant.

Quick Revision

Module-wise key points

- **Introduction to VLSI Testing and Fault Models:** Module 1 establishes the testing vocabulary: process, equipment, quality, abstraction, and the limits of each fault model.
- **Logic and Fault Simulation and Testability Measures:** Module 2 turns fault models into observable evidence through good-value simulation, faulty-value simulation, activation, propagation, and testability reasoning.
- **Test Generation for Combinational and Sequential Circuits:** Module 3 compares systematic and heuristic test-generation approaches for memoryless and stateful digital circuits.
- **Design for Testability, BIST and Fault Diagnosis:** Module 4 makes internal circuitry testable and uses structured response information to detect, localise, and diagnose faults.

Exam checklist

- Write the official terminology before adding explanation.
- Use a labelled diagram or step sequence when it improves the answer.
- Check conditions, boundaries, safety, hygiene, and documentation points.
- Separate official syllabus information from your own example.

Last-minute revision: Review the coverage table, formula bank, diagram library, and common-mistake notes inside every topic.

Glossary

Technical glossary

Term	Short meaning
------	---------------

Term	Short meaning
Explain VLSI Testing Process	The VLSI testing process converts a design specification into test requirements, derives test patterns, applies them to a device, captures the responses, compares the responses with expected values, and records pass/fail information. Testing may occur at wafer
Explain VLSI Testing Equipment	Automatic Test Equipment (ATE) supplies controlled input patterns, timing, power, and measurement functions to a device under test. A typical setup includes a tester controller, pattern memory, timing and signal-generation resources, power supplies, pin electr
Explain fault models	A fault model is a simplified logical representation of a physical defect. Common digital models include stuck-at-0 and stuck-at-1 faults, open faults, short or bridging faults, transistor-level defects, and delay faults. The stuck-at model assumes that a sign
Describe the relationship Among Fault Models.	Fault models are related by abstraction and by the defects they can represent. A physical defect may be represented by one or more logical faults, and a single logical model may cover several physical causes. For example, a short or transistor defect can somet
Explain Simulation for Design Verification and Test Evaluation	Simulation applies input vectors to a circuit model and computes the resulting internal and output values. In design verification, the objective is to check whether the implementation follows the specification. In test evaluation, the objective is to determine
Explain Modeling Circuits for Simulation	A simulation model represents gates, interconnections, storage elements, delays, and external signals at a selected abstraction level. A combinational model maps current inputs to outputs; a sequential model also represents state, clock events, reset, and feed
Explain the algorithms for true value and fault simulation	True-value simulation evaluates the fault-free circuit for each input vector. Fault simulation repeats or augments that computation for circuits containing targeted faults. A fault is activated when the good value at its site differs from the faulty value; it
Find the faults in the given logic circuit	Finding faults in a logic circuit follows a repeatable sequence. Identify the fault site and fault type, determine the good value required to activate it, choose values on side inputs that avoid masking, propagate the difference through a path, and observe the
Explain Combinational ATPG Algorithms	Combinational ATPG finds an input vector that activates a fault and propagates its effect to a primary output. Common reasoning steps are fault activation, forward implication, backward implication, path sensitization, and conflict resolution. Algorithms may u
Explain Sequential ATPG Algorithms	Sequential ATPG must control and observe circuit state as well as primary inputs. A test sequence may require an initialisation or reset operation, several capture cycles, and a final observation cycle. The algorithm searches through state transitions, uses ti
Explain Simulation based ATPG	Simulation-based ATPG uses circuit simulation to evaluate candidate test patterns and guide the search toward patterns that detect remaining faults. It may begin with random or deterministic patterns, simulate the fault list, drop detected faults, and generate

Term	Short meaning
Explain Genetic Algorithm Based ATPG	A Genetic Algorithm (GA) represents a candidate test pattern or sequence as a chromosome. A population of candidates is evaluated by a fitness function such as the number of faults detected, fault-effect propagation, or coverage gain. Selection retains stronge
Explain Testability basics and its analysis	Testability describes how easily a circuit can be controlled from available inputs and how easily internal effects can be observed at outputs. Controllability measures the effort needed to set a node to a required logic value; observability measures the effort
Explain BIST architecture	Built-In Self-Test (BIST) places test-pattern generation, response compaction or analysis, control, and pass/fail decision logic on or near the chip. In Random Logic BIST, a pseudo-random pattern generator stimulates logic and a signature register compacts res
Explain Fault Models for Diagnosis	Fault diagnosis goes beyond detection: it uses failing responses to identify or rank possible fault locations and types. Diagnostic fault models may include stuck-at, bridging, transition, open, or scan-chain faults according to the observed failure mechanism.

Official References and Resources

References listed in the supplied syllabus

1. T1 — Laung-Terng Wang, Cheng-Wen Wu and Xiaoqing Wen, “VLSI Test Principles and Architectures”, Elsevier, 2017.
2. T2 — Michael L. Bushnell and Vishwani D. Agrawal, “Essentials of Electronic Testing for Digital, Memory & Mixed-Signal VLSI Circuits”, Kluwer Academic Publishers, 2017.
3. R1 — Niraj K. Jha and Sandeep Gupta, “Testing of Digital Systems”, Cambridge University Press, 2017.
4. R2 — Parag K. Lala, “An Introduction to Logic Circuit Testing,” Morgan Claypool Publishers, 2009.
5. R3 — Parag K. Lala, “Digital Circuit Testing and Testability,” Academic Press, 1997.

Official learning resources listed

- https://onlinecourses.nptel.ac.in/noc21_ee39
- https://onlinecourses.nptel.ac.in/noc25_ee24/preview
- <https://www.geeksforgeeks.org/principles-in-digital-system-design/>

In-page Search

